Falas por Integrante v4

Startup XYZ — Gestão de Documentos para IA | Texto de cada um, alinhado ao deck v3 (16 slides, 4 integrantes)

Equipe: Athos, Gabriel, Fabricio, Lucas

> **Como usar**
> Cada integrante tem a própria página: dá para imprimir só a sua. Itálico laranja é indicação de cena e não se fala. Destaque azul é frase de impacto: diga com pausa antes e depois.

Integrante	Slides	Tempo	Palavras
Athos	Slides 1 a 4	3 minutos	~301
Fabricio	Slides 5 a 8	3 minutos	~385
Gabriel	Slides 9 a 13	3,5 minutos	~454
Lucas	Slides 14 a 16	4,5 minutos	~418

> **Tempo total**
> 1558 palavras, cerca de 11 minutos a 140 palavras por minuto, mais as ações da demo: em torno de 14 minutos, dentro da janela de 10 a 15 do guia.

> **O que mudou em relação às falas v3**
>
> - O trecho que era do Jhonatan (slides 5 a 8) passou para o Fabricio, inclusive a passagem "O Fabricio agora mostra" no fim da fala do Athos e a apresentação da equipe na abertura.
> - Conferido com o deck v3 em HTML: entrou na demo o sexto passo (auditoria com CloudTrail e logs da API), que estava no slide 14 e faltava na fala do Lucas.

Athos 3 minutos (~301 palavras)

Slides 1 a 4 — Capa, Entendimento do Problema, Levantamento de Requisitos e Visão Geral da Solução.

[olhar para a banca, não para o slide, ao dizer o número]

Bom dia. Nós somos Athos, Gabriel, Fabricio e Lucas. Antes de mostrar qualquer diagrama, quero deixar um número com vocês: ***cinquenta mil***. É a quantidade de documentos que a Startup XYZ recebe por mês, de clientes que confiam a ela arquivos que não podem ser perdidos nem vistos por outra empresa.

[transição de slide: Entendimento do Problema]

O problema aqui não é só guardar arquivo. É guardar o ativo que essa empresa vai usar para treinar a próxima geração da própria inteligência artificial.

A XYZ é um SaaS que extrai dados de PDFs e imagens com IA. Quando ela cresce, aparecem três problemas ao mesmo tempo. Se todo upload passar pelo servidor, o servidor vira gargalo. Com milhares de clientes, o cliente A nunca pode ver o documento do cliente B. E apagar o histórico não é opção, mas guardar tudo em armazenamento rápido faz o custo subir todo mês.

[transição de slide: Levantamento de Requisitos]

Separamos os requisitos em dois grupos. Funcionais: autenticar o usuário, receber PDF e imagem sem gargalo, isolar os documentos por cliente, permitir download seguro e preservar o histórico por anos. Não funcionais: escalar com cinquenta mil arquivos por mês, segurança e privacidade pensando na LGPD, custo proporcional ao uso, alta disponibilidade sem administrar servidores e rastreabilidade de quem acessou o quê.

Não construímos um repositório de arquivos. Construímos uma estratégia de valor ao longo do tempo.

[transição de slide: Visão Geral da Solução]

Em linguagem simples, a solução tem quatro passos. O usuário faz login e recebe um token que diz quem ele é. A API confere esse token e decide o que ele pode acessar. O sistema devolve uma chave temporária que abre só aquele arquivo, por poucos minutos. E o arquivo vai direto para o armazenamento, que depois de um ano o move sozinho para uma camada muito mais barata, sem apagar nada. O Fabricio agora mostra como cada peça faz isso.

Fabricio 3 minutos (~385 palavras)

Slides 5 a 8 — Diagrama de Arquitetura, Serviços AWS, Justificativa Técnica e Atendimento aos Requisitos.

[apontar o diagrama da esquerda para a direita, seguindo os números das setas]

Obrigado, Athos. A forma mais simples de resolver isso seria um servidor ligado o tempo todo esperando os uploads. Foi exatamente o que decidimos não fazer. Cinquenta mil arquivos por mês dão pouco mais de um por minuto: um servidor ligado vinte e quatro horas ficaria ocioso quase o tempo todo e ainda seria o gargalo nos picos.

[seta 1: Cliente → Cognito]

O fluxo começa no Amazon Cognito. É ele que autentica o usuário e emite um token JWT. Usamos o Cognito para não construir login do zero.

[seta 2: Cliente → API Gateway]

Com esse token, o app chama o Amazon API Gateway. O Gateway não autentica ninguém: ele valida o token do Cognito e limita o número de requisições. Sem token válido a chamada morre ali, e a Lambda nem executa.

[setas 3 e 4: Lambda e STS]

Depois vem a AWS Lambda, e aqui está o que diferencia a nossa solução. A Lambda não recebe o arquivo. Ela lê quem é o usuário no próprio token, pede ao AWS STS uma credencial temporária que só enxerga a pasta desse usuário e usa essa credencial para assinar uma URL válida por quinze minutos.

O arquivo vai direto do cliente para o Amazon S3. A Lambda nunca toca no binário, então não há limite de payload nem gargalo no backend.

[apontar para o bucket S3 e para os quadros de baixo]

O S3 é o coração da solução. Os objetos são cifrados com uma chave nossa no AWS KMS, e CloudWatch e CloudTrail registram o que acontece. O GuardDuty aparece no desenho como adicional opcional: ele escaneia cada upload, mas vem desligado, e o Gabriel vai explicar por quê.

[transição de slide: Justificativa Técnica]

[apontar a tabela linha a linha]

Cada escolha tem uma alternativa descartada. Serverless em vez de EC2, porque não há servidor ocioso. S3 em vez de EFS, porque os documentos são gravados uma vez e não editados. Um bucket com uma pasta por cliente em vez de um bucket por cliente: a AWS até permite milhares, mas seria uma política, um ciclo de vida e uma chave para cada um. E São Paulo em vez de Virgínia: Virgínia sairia cerca de quarenta e cinco por cento mais barata, mas dados de clientes brasileiros ficam no Brasil, sem transferência internacional pela LGPD.

[transição de slide: Atendimento aos Requisitos]

Cada requisito que o Athos apresentou tem um componente concreto respondendo por ele. A tabela mostra isso linha por linha.

Agora o Gabriel mostra como essa arquitetura se comporta quando cresce, quando algo falha e quanto ela custa.

Gabriel 3,5 minutos (~454 palavras)

Slides 9 a 13 — Escalabilidade, Resiliência, Segurança, Custos e Backup e Recuperação.

[pausa de um segundo antes de começar, tom mais firme]

Vou responder três perguntas antes que vocês precisem fazer: o que acontece se o volume crescer dez vezes, o que acontece se algo falhar, e quanto isso custa.

Crescimento. Cinquenta mil documentos por mês são pouco mais de um por minuto; dez vezes mais são doze por minuto. Nossa API aceita dez requisições por segundo por rota, e a conta aceita dez mil. Não existe servidor para redimensionar, e os limites que existem são cotas de conta, ajustáveis com um pedido à AWS. Na demo, cada autorização levou cerca de duzentos e cinquenta milissegundos.

Crescer dez vezes não exige redesenhar a solução. Exige, no máximo, um pedido de aumento de cota.

[transição de slide: Resiliência]

Falhas. Não há instância para cair: Lambda, API Gateway e Cognito rodam em várias zonas, e pelos SLAs da AWS a disponibilidade composta fica em torno de noventa e nove vírgula sete por cento. O S3 grava cada objeto em pelo menos três zonas antes de confirmar o upload, por isso o RPO é praticamente zero. Não replicamos para outra região porque o Brasil tem uma só, e copiar dados pessoais para fora do país é transferência internacional pela LGPD.

[transição de slide: Segurança]

Segurança em três camadas. Identidade: a Lambda lê o usuário do token do Cognito, nunca do que o cliente envia. Rede: não usamos VPC de propósito, porque nenhum componente precisa de rede privada; o controle está no bucket fechado ao público, na exigência de TLS e no limite de requisições. Dados: criptografia com KMS e registro no CloudTrail de cada credencial emitida.

[transição de slide: Custos]

[apontar para a tabela de custos]

Custo. Em São Paulo, a conta fica em torno de vinte e seis dólares por mês no primeiro ano e trinta e cinco no terceiro. Mais de dois terços disso é o primeiro ano de documentos no S3 Standard; Lambda e API somam centavos. Sem o ciclo de vida, o terceiro ano custaria setenta e quatro dólares por mês.

O armazenamento fica noventa e dois por cento mais barato no momento em que o documento deixa de ser operação e vira histórico.

O GuardDuty ficou como adicional porque custaria uns vinte dólares por mês, quase o mesmo que toda a infraestrutura do primeiro ano. Quem precisa de antimalware contrata; quem não precisa não paga. E a próxima alavanca já está mapeada: trinta dias em Standard e depois Glacier Instant corta quase metade do custo em três anos.

[transição de slide: Backup e Recuperação]

Por fim, recuperação: o versionamento desfaz qualquer sobrescrita, e a política do bucket só deixa a conta root apagar uma versão de vez. E um achado nosso: o template ainda não tem DeletionPolicy Retain na chave KMS; se alguém excluir a pilha, a chave entra em exclusão e os documentos ficariam ilegíveis. Já está nos próximos passos. O Lucas agora mostra tudo isso funcionando.

Lucas 4,5 minutos (~418 palavras)

Slides 14 a 16 — Demonstração ao vivo (3 min), Evolução e Conclusão (1,5 min).

[tela: página da demo na aba do cliente-a, já logado; console da Lambda aberto em outra aba]

Até aqui vocês viram o desenho. Agora vão ver a arquitetura rodando, na nossa conta AWS em São Paulo.

[clicar em "Chamar a API sem login"]

Primeiro, alguém de fora chama a nossa API sem login: resposta 401. Com um token forjado, 401 de novo. Isso para no API Gateway; a Lambda nem executa.

[escolher contrato-cliente-a.pdf e clicar em Enviar]

Agora o cliente A envia um contrato. A API devolveu a URL e o navegador gravou direto no S3: resposta 204. Reparem na chave: o arquivo caiu na pasta usuario- seguida do identificador desse usuário no Cognito, com um nome gerado pelo servidor.

[colar no campo 4 a chave do arquivo do cliente-b e clicar em Tentar baixar]

Agora o ataque. O cliente A descobriu o nome de um arquivo do cliente B e pede o download. A aplicação recusa: 403, chave fora do seu prefixo. Mas e se o nosso código tivesse um bug e esquecesse essa checagem?

[console: APP_CHECK = false, Salvar; voltar à página e clicar de novo em Tentar baixar]

Desliguei a checagem da aplicação para simular o bug. A Lambda assinou a URL, e o S3 respondeu 403 AccessDenied. A credencial que assinou essa URL foi emitida pelo STS só para a pasta do cliente A. ***O isolamento não depende de o código lembrar de checar.***

[console: voltar APP_CHECK para true e salvar]

[na lista, clicar em Baixar no documento DEEP_ARCHIVE e depois em Restaurar]

Por último, o tempo. Este documento está em Deep Archive, como estaria depois de um ano. O download é recusado com a mensagem de arquivado, e o cliente pede a restauração: 202. Em até doze horas o arquivo volta. Ninguém apagou nada.

[apontar para o CloudTrail ou para os logs da API]

E o sexto passo é a auditoria: o CloudTrail e os logs da API registraram cada credencial emitida e cada acesso que vocês viram agora.

[transição de slide: Evolução]

Tudo o que vocês viram resolve o problema de hoje. O que vem depois: uma fila SQS entre o upload e a extração de IA, para que um pico de envios não sobrecarregue o processamento; a regra de ciclo de vida com Glacier Instant que o Gabriel mostrou; Object Lock para tornar o histórico imutável; e o Macie para achar dados pessoais antes de usar o histórico no treino da IA.

[transição de slide: Conclusão]

[olhar para todos os avaliadores]

Para fechar, volto ao número do Athos: cinquenta mil documentos por mês que não podem ser perdidos nem vistos por outro cliente, com um custo que não pode explodir com o tempo.

A resposta não é a arquitetura mais complexa. É a mais honesta com o problema: cada serviço faz o que faz de melhor, o armazenamento fica noventa e dois por cento mais barato quando o dado vira histórico, e o isolamento entre clientes foi provado ao vivo, não só no papel.

Segurança desenhada na arquitetura, custo sob controle e nenhum documento perdido. Obrigado, e ficamos à disposição para as perguntas.